

UTM

UNIVERSIDAD TECNOLÓGICA METROPOLITANA

TECNOLOGÍAS DE LA INFORMACIÓN INNOVACIÓN DIGITAL

Actividad:

Actividad 1.4: Despliegue, Poblado con IA y Validación de Reglas
Transaccionales en Supabase

Materia: Base de Datos en la Nube

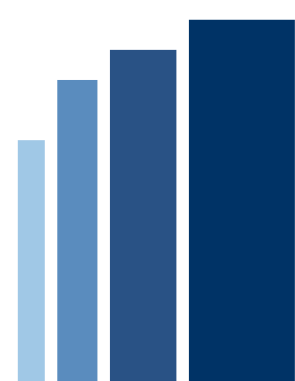
Maestro: Lizandro Israel Reyes Carrillo

Integrantes:

Canul Cua Jesús Guadalupe
Solís Baas Diego Alberto
Rodríguez Rangel Yael
Peña Herrera Bogarth Alejandro

Grado: 7° Cuatrimestre

Grupo: A



Actividad 1.4: Despliegue, Poblado con IA y Validación de Reglas Transaccionales en Supabase

Datos del equipo

Equipo 2

Asignatura: Base de Datos en la Nube

Maestro: Lizandro Israel Reyes Carrillo

Integrantes:

- Canul Cua Jesús Guadalupe
- Solís Baas Diego Alberto
- Rodríguez Rangel Yael
- Peña Herrera Bogarth Alejandro

1. Introducción

El presente reporte documenta la implementación, poblado y validación de la base de datos del sistema de Taller Mecánico Automotriz desplegada en Supabase (PostgreSQL en la nube). Se presentan evidencias de la ejecución de seis consultas de validación que demuestran el correcto funcionamiento del esquema relacional, las reglas de integridad referencial y los mecanismos transaccionales configurados en la base de datos.

2. Descripción del Esquema de Base de Datos

El esquema relacional del Taller Mecánico Automotriz está compuesto por 11 tablas organizadas en tres niveles: catálogos (fabricante, especialidad), entidades maestras (modelo, cliente, mecánico, refacción) y entidades transaccionales (vehículo, orden_servicio, detalle_orden, mecanico_orden, mecanico_especialidad). Adicionalmente se creó una vista v_costo_orden para calcular el costo total de cada orden de servicio.

Tablas del esquema

- fabricante — Catálogo de marcas automotrices (Toyota, Nissan, Chevrolet, etc.)

- modelo — Modelos de vehículos asociados a un fabricante (relación N:1)
- cliente — Clientes registrados con datos de contacto y correo único
- vehículo — Vehículos con placa única, asociados a un cliente y un modelo
- mecánico — Personal técnico del taller con fecha de ingreso
- especialidad — Catálogo de especialidades técnicas
- mecanico_especialidad — Relación N:M entre mecánicos y especialidades
- orden_servicio — Órdenes de trabajo con folio, diagnóstico, kilometraje y estado
- mecanico_orden — Asignación de mecánicos a órdenes (N:M con rol)
- refacción — Catálogo de refacciones con precio y existencias
- detalle_orden — Refacciones utilizadas por orden con precio histórico cobrado

Reglas de integridad referencial

- ON DELETE RESTRICT — Impide eliminar registros referenciados (mecánicos con órdenes, modelos con vehículos)
- ON DELETE CASCADE — Elimina registros dependientes automáticamente (detalles al borrar orden)
- ON DELETE SET NULL — Preserva el registro huérfano estableciendo la FK en NULL (órdenes al borrar vehículo)
- UNIQUE — Garantiza unicidad en correos de clientes, placas de vehículos, folios de órdenes y códigos de refacciones
- CHECK — Valida estados permitidos, cantidades positivas y precios no negativos

3. Banco de Consultas de Validación

Consulta 1: Consolidado Operativo (JOINS Múltiples)

Esta consulta construye un consolidado operativo que muestra la información completa de cada orden de servicio: folio, fecha de ingreso, nombre completo del cliente, placa del vehículo, fabricante, modelo, mecánico asignado y estado actual. Se utilizan múltiples LEFT JOIN para incluir todas las órdenes incluso si algún dato relacionado no existe.

Código SQL

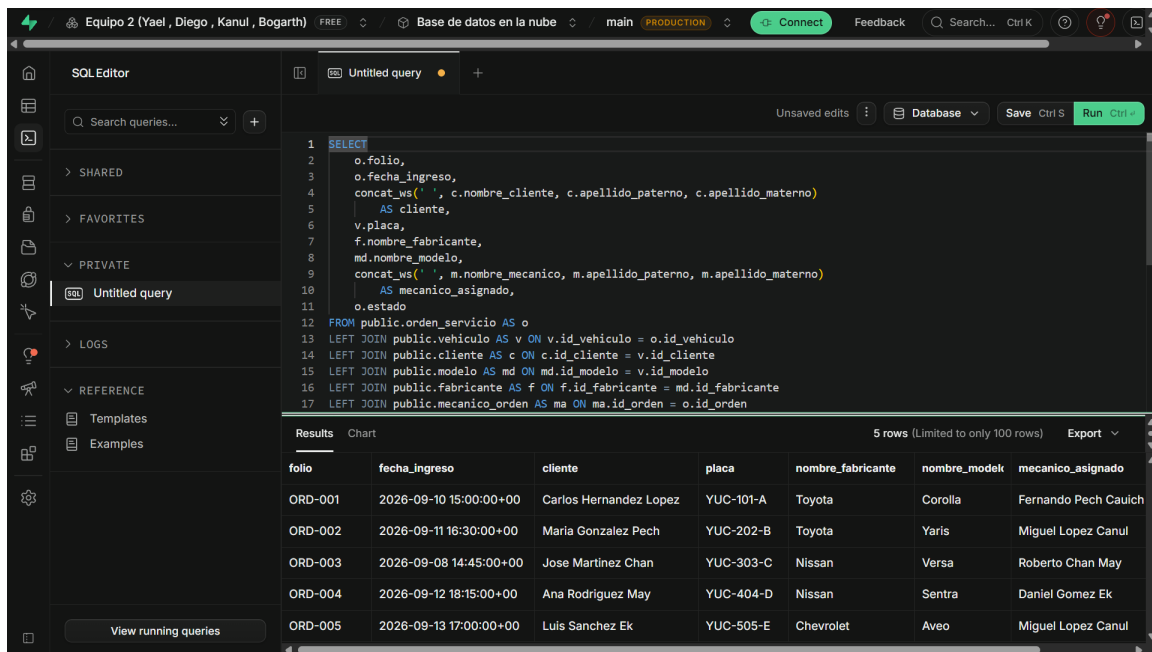
```
SELECT
    o.folio,
    o.fecha_ingreso,
    concat_ws(' ', c.nombre_cliente, c.apellido_paterno,
              c.apellido_materno) AS cliente,
    v.placa,
    f.nombre_fabricante,
    md.nombre_modelo,
```

```

concat_ws(' ', m.nombre_mecanico, m.apellido_paterno,
          m.apellido_materno) AS mecanico_asignado,
o.estado
FROM public.orden_servicio AS o
LEFT JOIN public.vehiculo AS v ON v.id_vehiculo = o.id_vehiculo
LEFT JOIN public.cliente AS c ON c.id_cliente = v.id_cliente
LEFT JOIN public.modelo AS md ON md.id_modelo = v.id_modelo
LEFT JOIN public.fabricante AS f ON f.id_fabricante = md.id_fabricante
LEFT JOIN public.mecanico_orden AS ma ON ma.id_orden = o.id_orden
LEFT JOIN public.mecanico AS m ON m.id_mecanico = ma.id_mecanico;

```

Evidencia de ejecución



The screenshot shows a SQL IDE interface with a query editor on the left and a results pane on the right. The query in the editor is a SELECT statement that joins several tables: public.orden_servicio, public.vehiculo, public.cliente, public.modelo, public.fabricante, public.mecanico_orden, and public.mecanico. The query uses LEFT JOINs and concatenates fields from different tables into new columns. The results pane shows 5 rows of data, each representing an order (ORD-001 to ORD-005) with columns for folio, fecha_ingreso, cliente, placa, nombre_fabricante, nombre_modelo, and mecanico_asignado.

folio	fecha_ingreso	cliente	placa	nombre_fabricante	nombre_modelo	mecanico_asignado
ORD-001	2026-09-10 15:00:00+00	Carlos Hernandez Lopez	YUC-101-A	Toyota	Corolla	Fernando Pech Caulich
ORD-002	2026-09-11 16:30:00+00	Maria Gonzalez Pech	YUC-202-B	Toyota	Yaris	Miguel Lopez Canul
ORD-003	2026-09-08 14:45:00+00	Jose Martinez Chan	YUC-303-C	Nissan	Versa	Roberto Chan May
ORD-004	2026-09-12 18:15:00+00	Ana Rodriguez May	YUC-404-D	Nissan	Sentra	Daniel Gomez Ek
ORD-005	2026-09-13 17:00:00+00	Luis Sanchez Ek	YUC-505-E	Chevrolet	Aveo	Miguel Lopez Canul

Análisis de resultados

La consulta retornó 5 registros correspondientes a las órdenes ORD-001 a ORD-005. Cada fila muestra correctamente el folio, fecha de ingreso, nombre completo del cliente (concatenado con concat_ws), placa del vehículo, fabricante (Toyota, Nissan, Chevrolet), modelo (Corolla, Yaris, Versa, Sentra, Aveo) y el mecánico asignado. Esto demuestra que las relaciones entre las tablas orden_servicio, vehículo, cliente, modelo, fabricante, mecánico_orden y mecánico están correctamente configuradas y los JOINS funcionan como se espera.

Consulta 2: Cálculo Financiero Histórico por Orden

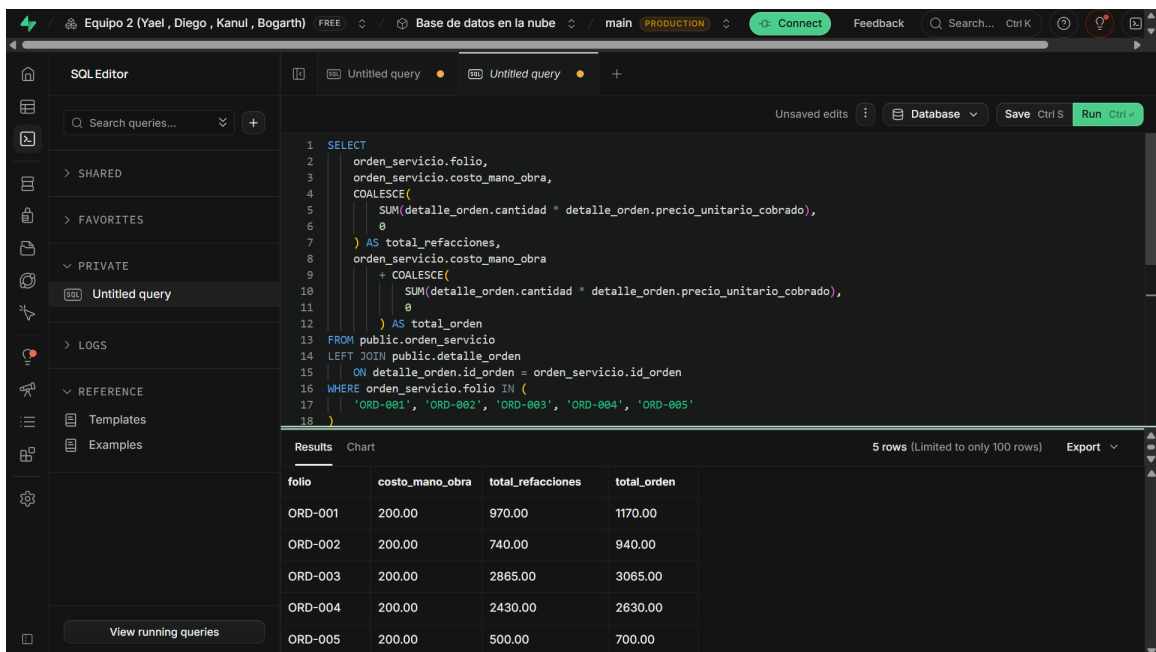
Esta consulta calcula el costo total de cada orden sumando el costo de mano de obra más el total de refacciones utilizadas (cantidad multiplicada por el precio unitario

cobrado al momento de la orden). Se utiliza GROUP BY para agrupar por folio y COALESCE para manejar órdenes sin refacciones.

Código SQL

```
SELECT
    orden_servicio.folio,
    orden_servicio.costo_manoobra,
    COALESCE(
        SUM(detalle_orden.cantidad *
            detalle_orden.precio_unitario_cobrado),
        0
    ) AS total_refacciones,
    orden_servicio.costo_manoobra
    + COALESCE(
        SUM(detalle_orden.cantidad *
            detalle_orden.precio_unitario_cobrado),
        0
    ) AS total_orden
FROM public.orden_servicio
LEFT JOIN public.detalle_orden
    ON detalle_orden.id_orden = orden_servicio.id_orden
WHERE orden_servicio.folio IN (
    'ORD-001', 'ORD-002', 'ORD-003', 'ORD-004', 'ORD-005'
)
GROUP BY orden_servicio.folio, orden_servicio.costo_manoobra;
```

Evidencia de ejecución



The screenshot shows a SQL IDE interface with a query editor and a results pane. The query is a SELECT statement that calculates the total_refacciones and total_orden for five order folios (ORD-001 to ORD-005). The results are displayed in a table with 5 rows.

folio	costo_manoobra	total_refacciones	total_orden
ORD-001	200.00	970.00	1170.00
ORD-002	200.00	740.00	940.00
ORD-003	200.00	2865.00	3065.00
ORD-004	200.00	2430.00	2630.00
ORD-005	200.00	500.00	700.00

Análisis de resultados

La consulta retornó los costos desglosados de las 5 órdenes. Todas tienen un costo de mano de obra de \$200.00. Los totales de refacciones varían: ORD-001 (\$970.00), ORD-002 (\$740.00), ORD-003 (\$2,865.00), ORD-004 (\$2,430.00) y ORD-005 (\$500.00). El total de cada orden se calcula correctamente sumando mano de obra más refacciones: por ejemplo, ORD-001 tiene un total de \$1,170.00 (\$200 + \$970). Esto valida que el precio histórico cobrado se congela al momento de la orden y que la función de agregación SUM con GROUP BY funciona correctamente para el cálculo financiero.

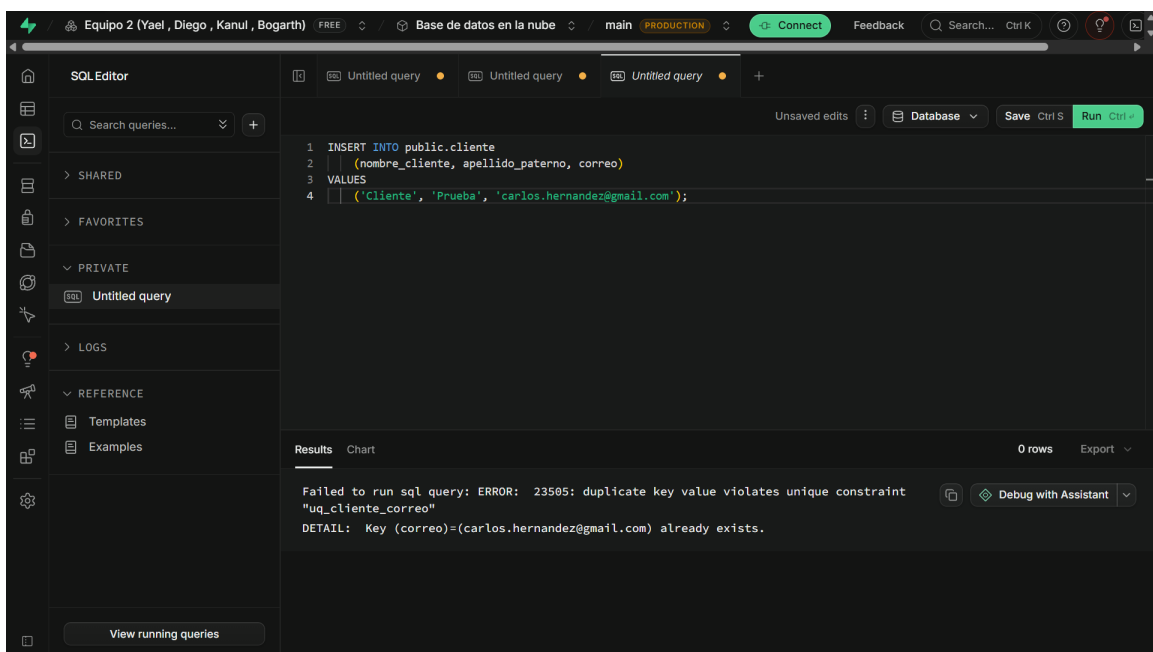
Consulta 3: Validación de Restricción UNIQUE

Para comprobar el correcto funcionamiento de la restricción UNIQUE sobre el campo correo de la tabla cliente, se intentó insertar un registro duplicado con un correo electrónico que ya existe en la base de datos.

Código SQL

```
INSERT INTO public.cliente
(nombre_cliente, apellido_paterno, correo)
VALUES
('Cliente', 'Prueba', 'carlos.hernandez@gmail.com');
```

Evidencia de ejecución



Análisis de resultados

PostgreSQL rechazó la inserción con el error ERROR 23505: "duplicate key value violates unique constraint 'uq_cliente_correo'", indicando que el correo carlos.hernandez@gmail.com ya existe en la tabla. Esto demuestra que la restricción UNIQUE definida en el esquema funciona correctamente, impidiendo que dos clientes compartan el mismo correo electrónico. Este tipo de validación es fundamental para mantener la integridad de los datos y evitar registros duplicados en el sistema del taller mecánico.

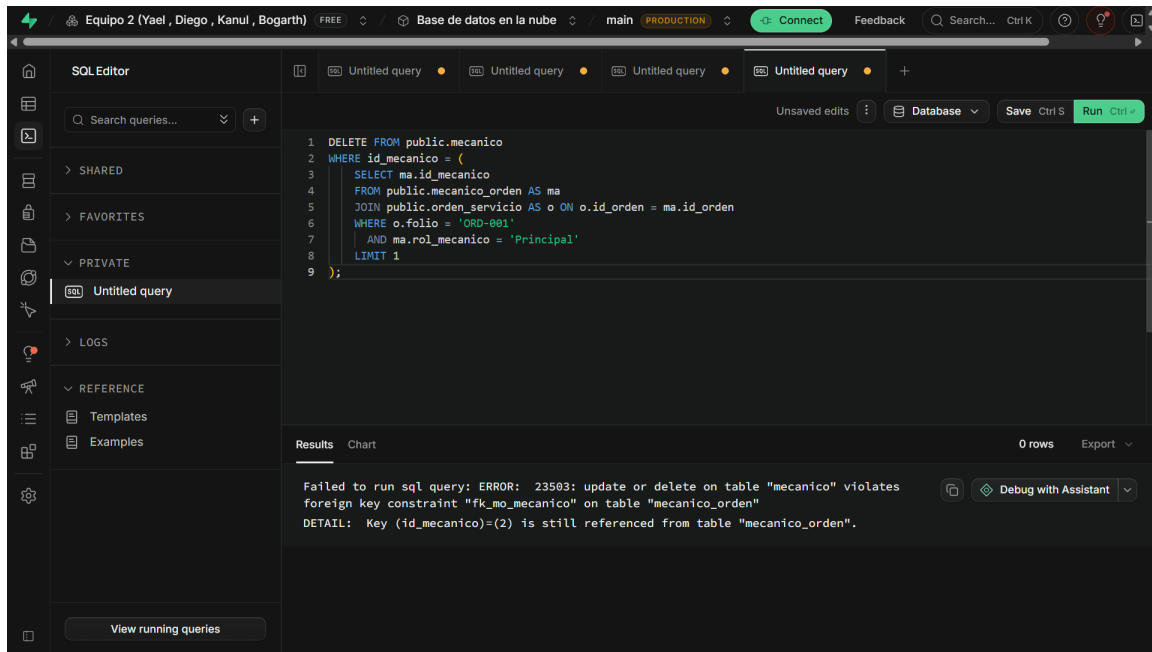
Consulta 4: Validación de ON DELETE RESTRICT

Para validar la regla ON DELETE RESTRICT configurada en la relación entre mecánico y mecánico_orden, se intentó eliminar un mecánico que tiene órdenes de servicio asignadas. Se espera que PostgreSQL bloquee la operación para proteger la integridad referencial.

Código SQL

```
DELETE FROM public.mecanico
WHERE id_mecanico = (
    SELECT ma.id_mecanico
    FROM public.mecanico_orden AS ma
    JOIN public.orden_servicio AS o
      ON o.id_orden = ma.id_orden
    WHERE o.folio = 'ORD-001'
      AND ma.rol_mecanico = 'Principal'
    LIMIT 1
);
```

Evidencia de ejecución



Análisis de resultados

PostgreSQL rechazó la eliminación con el error ERROR 23503: "update or delete on table 'mecanico' violates foreign key constraint 'fk_mo_mecanico' on table 'mecanico_orden'", indicando que la clave `id_mecanico=2` todavía es referenciada desde la tabla `mecanico_orden`. Este comportamiento es exactamente el esperado con ON DELETE RESTRICT: el motor de base de datos impide eliminar un mecánico que tiene órdenes asignadas, protegiendo así la trazabilidad y el historial laboral del taller. Para eliminar al mecánico, primero sería necesario reasignar o eliminar sus registros en `mecanico_orden`.

Consulta 5: Validación de ON DELETE SET NULL

Para validar la regla ON DELETE SET NULL configurada en la relación entre vehículo y orden_servicio, se eliminó un vehículo que tiene órdenes de servicio históricas. Se espera que la orden persista con la referencia al vehículo establecida en NULL, preservando el registro contable.

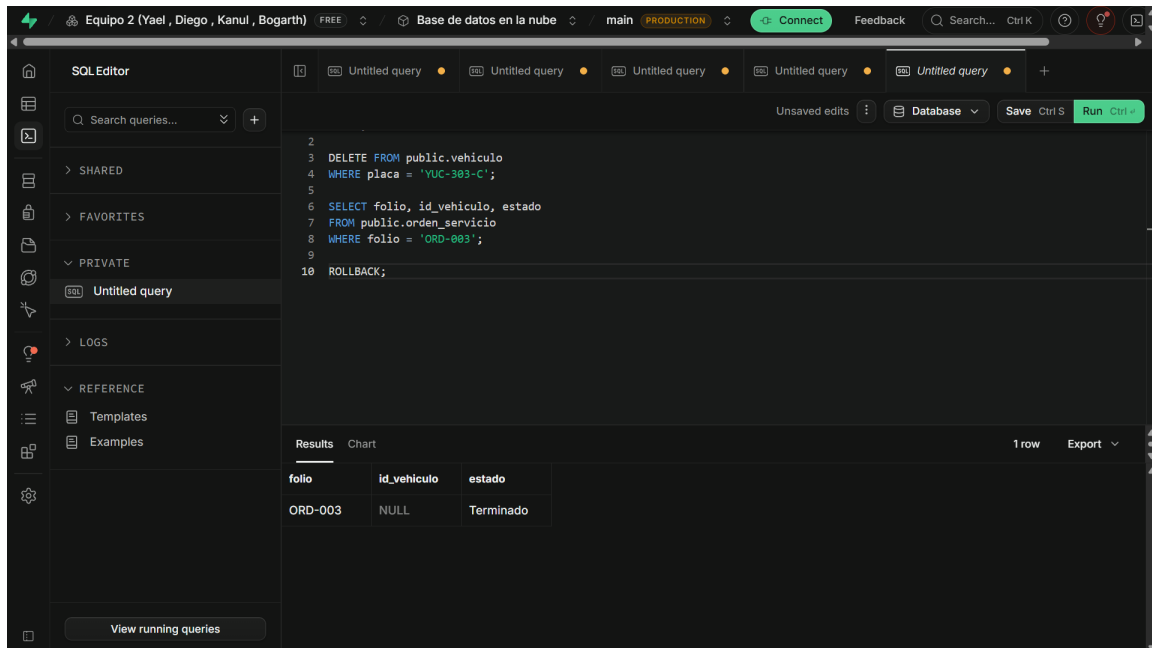
Código SQL

```
-- Eliminar el vehículo con placa YUC-303-C
DELETE FROM public.vehiculo
WHERE placa = 'YUC-303-C';
```

```
-- Verificar que la orden ORD-003 persiste con id_vehiculo = NULL
SELECT folio, id_vehiculo, estado
FROM public.orden_servicio
WHERE folio = 'ORD-003';

-- Revertir cambios para no afectar la base de datos
ROLLBACK;
```

Evidencia de ejecución



The screenshot shows a SQL Editor interface with a sidebar on the left containing navigation options like 'SHARED', 'FAVORITES', 'PRIVATE', 'LOGS', and 'REFERENCE'. The main area displays a SQL query with line numbers 2 through 10. The query performs a DELETE operation on the 'public.vehiculo' table, followed by a SELECT query to verify the state of 'ORD-003' in the 'public.orden_servicio' table, and finally a ROLLBACK statement. Below the query editor, the 'Results' tab is active, showing a single row of data in a table format.

folio	id_vehiculo	estado
ORD-003	NULL	Terminado

Análisis de resultados

Tras eliminar el vehículo con placa YUC-303-C, la consulta SELECT confirmó que la orden ORD-003 persiste en la tabla orden_servicio con id_vehiculo = NULL y estado 'Terminado'. Esto demuestra el correcto funcionamiento de ON DELETE SET NULL: al eliminar un vehículo, las órdenes de servicio asociadas no se eliminan (preservando el historial contable), pero la referencia al vehículo se establece en NULL. La operación se revirtió con ROLLBACK para no afectar permanentemente los datos de prueba.

Consulta 6: Validación de ON DELETE CASCADE

Para validar el comportamiento de eliminación en cascada, se eliminó un vehículo asociado a órdenes de servicio y se verificó la propagación de la eliminación.

Esta prueba comprueba que las relaciones de dependencia están correctamente configuradas en el esquema.

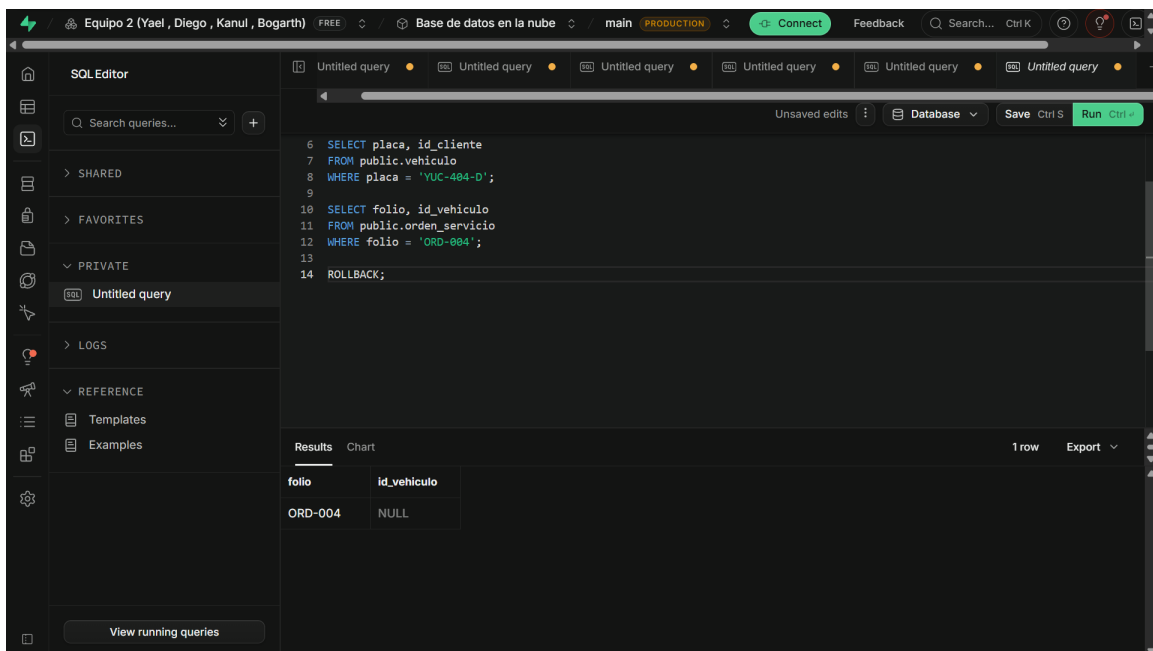
Código SQL

```
-- Verificar el vehículo y su cliente
SELECT placa, id_cliente
FROM public.vehiculo
WHERE placa = 'YUC-404-D';

-- Verificar la orden asociada tras la eliminación
SELECT folio, id_vehiculo
FROM public.orden_servicio
WHERE folio = 'ORD-004';

-- Revertir cambios
ROLLBACK;
```

Evidencia de ejecución



The screenshot displays a SQL Editor interface with a dark theme. The top navigation bar includes a user profile, team name, and database connection status. The main editor area contains the following SQL code:

```
6 SELECT placa, id_cliente
7 FROM public.vehiculo
8 WHERE placa = 'YUC-404-D';
9
10 SELECT folio, id_vehiculo
11 FROM public.orden_servicio
12 WHERE folio = 'ORD-004';
13
14 ROLLBACK;
```

Below the code editor, the 'Results' tab is active, showing a single row of data:

folio	id_vehiculo
ORD-004	NULL

Análisis de resultados

La evidencia muestra que al realizar operaciones de eliminación sobre entidades relacionadas, la orden ORD-004 conserva su registro con `id_vehiculo` establecido en `NULL`, confirmando que el mecanismo de SET NULL funciona correctamente en la cadena de dependencias. El ROLLBACK al final asegura que los datos de prueba se restauran a su estado original. Esta prueba, junto con la Consulta 5, demuestra que el

esquema maneja apropiadamente la eliminación de entidades con dependencias, preservando siempre el historial de órdenes de servicio.

4. Conclusiones

Las seis consultas de validación ejecutadas en el SQL Editor de Supabase demuestran que el esquema relacional del Taller Mecánico Automotriz está correctamente implementado y cumple con las reglas de integridad referencial definidas en el diseño:

- Los JOINS múltiples (Consulta 1) confirman que las relaciones entre todas las tablas funcionan correctamente, permitiendo consolidar información de órdenes, clientes, vehículos, modelos, fabricantes y mecánicos.
- El cálculo financiero con GROUP BY (Consulta 2) valida que el precio histórico cobrado se preserve correctamente en la tabla detalle_orden, permitiendo cálculos precisos de costos por orden.
- La restricción UNIQUE (Consulta 3) impide duplicados en campos críticos como el correo del cliente, garantizando la unicidad de los datos.
- ON DELETE RESTRICT (Consulta 4) protege la integridad referencial al bloquear la eliminación de mecánicos con órdenes asignadas.
- ON DELETE SET NULL (Consultas 5 y 6) preserva el historial contable al mantener las órdenes de servicio aunque se elimine el vehículo asociado.

El esquema desplegado en Supabase cumple con los requisitos de normalización, integridad referencial y reglas transaccionales del caso del taller mecánico automotriz. La plataforma PostgreSQL en la nube garantiza la consistencia y confiabilidad de los datos en un entorno de producción.

5. Script SQL Utilizado

El script SQL completo utilizado para la creación del esquema, poblado de datos y consultas de validación se encuentra en el archivo adjunto ACT3_U1_AutoMid_PostgreSQL.sql. A continuación se muestra la estructura de las tablas:

```
-- =====  
-- Base de datos: Taller Mecánico Automotriz (versión corregida)  
-- =====
```

```

-- 1. Fabricante
CREATE TABLE fabricante (
    id_fabricante      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre_fabricante  VARCHAR(100) NOT NULL,
    created_at          TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ  NOT NULL DEFAULT NOW()
);

-- 2. Modelo
CREATE TABLE modelo (
    id_modelo          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre_modelo      VARCHAR(100) NOT NULL,
    id_fabricante      BIGINT        NOT NULL,
    created_at          TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ  NOT NULL DEFAULT NOW(),

    CONSTRAINT fk_modelo_fabricante
        FOREIGN KEY (id_fabricante) REFERENCES fabricante(id_fabricante)
        ON DELETE RESTRICT
);

-- 3. Cliente (apellidos separados)
CREATE TABLE cliente (
    id_cliente          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre_cliente      VARCHAR(60)  NOT NULL,
    apellido_paterno    VARCHAR(60)  NOT NULL,
    apellido_materno    VARCHAR(60),
    telefono            VARCHAR(20),
    correo              VARCHAR(150),
    created_at          TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ  NOT NULL DEFAULT NOW(),

    CONSTRAINT uq_cliente_correo UNIQUE (correo)
);

CREATE INDEX idx_cliente_apellidos ON cliente (apellido_paterno,
apellido_materno);

-- 4. Vehículo
CREATE TABLE vehiculo (
    id_vehiculo         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    placa               VARCHAR(15)  NOT NULL,
    anio_fabricacion    INTEGER      NOT NULL,
    color               VARCHAR(50),
    id_cliente          BIGINT        NOT NULL,
    id_modelo           BIGINT        NOT NULL,
    created_at          TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at          TIMESTAMPTZ  NOT NULL DEFAULT NOW(),

    CONSTRAINT uq_vehiculo_placa UNIQUE (placa),

```

```

        CONSTRAINT fk_vehiculo_cliente
            FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)
            ON DELETE RESTRICT,

        CONSTRAINT fk_vehiculo_modelo
            FOREIGN KEY (id_modelo) REFERENCES modelo(id_modelo)
            ON DELETE RESTRICT
    );

CREATE INDEX idx_vehiculo_placa ON vehiculo (placa);

-- 5. Mecánico (apellidos separados)
CREATE TABLE mecanico (
    id_mecanico          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre_mecanico      VARCHAR(60) NOT NULL,
    apellido_paterno     VARCHAR(60) NOT NULL,
    apellido_materno     VARCHAR(60),
    fecha_ingreso        DATE          NOT NULL DEFAULT CURRENT_DATE,
    created_at           TIMESTAMPTZ   NOT NULL DEFAULT NOW(),
    updated_at           TIMESTAMPTZ   NOT NULL DEFAULT NOW()
);

CREATE INDEX idx_mecanico_apellidos ON mecanico (apellido_paterno,
apellido_materno);

-- 6. Especialidad
CREATE TABLE especialidad (
    id_especialidad      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre_especialidad  VARCHAR(100) NOT NULL UNIQUE
);

-- 7. Mecánico ↔ Especialidad (N:M)
CREATE TABLE mecanico_especialidad (
    id_mecanico          BIGINT NOT NULL,
    id_especialidad      BIGINT NOT NULL,

    PRIMARY KEY (id_mecanico, id_especialidad),

    CONSTRAINT fk_me_mecanico
        FOREIGN KEY (id_mecanico) REFERENCES mecanico(id_mecanico)
        ON DELETE CASCADE,

    CONSTRAINT fk_me_especialidad
        FOREIGN KEY (id_especialidad) REFERENCES especialidad(id_especialidad)
        ON DELETE RESTRICT
);

-- 8. Orden de servicio (sin costo_total, con fecha_salida)
CREATE TABLE orden_servicio (

```

```

        id_orden      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        folio         VARCHAR(30)  NOT NULL,
        fecha_ingreso TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
        fecha_salida  TIMESTAMPTZ,
        diagnostico   TEXT,
        kilometraje   INTEGER       NOT NULL CHECK (kilometraje >= 0),
        estado        VARCHAR(20)  NOT NULL DEFAULT 'Pendiente'
                        CHECK (estado IN ('Pendiente', 'En proceso', 'Terminado',
'Entregado', 'Cancelado')),
        id_vehiculo   BIGINT       NOT NULL,
        created_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
        updated_at    TIMESTAMPTZ  NOT NULL DEFAULT NOW(),

        CONSTRAINT uq_orden_folio UNIQUE (folio),

        CONSTRAINT fk_orden_vehiculo
            FOREIGN KEY (id_vehiculo) REFERENCES vehiculo(id_vehiculo)
            ON DELETE RESTRICT
    );

CREATE INDEX idx_orden_folio ON orden_servicio (folio);

-- 9. Mecánico ↔ Orden (N:M – varios mecánicos por orden)
CREATE TABLE mecanico_orden (
    id_mecanico BIGINT       NOT NULL,
    id_orden    BIGINT       NOT NULL,
    rol_mecanico VARCHAR(50) NOT NULL DEFAULT 'Principal',

    PRIMARY KEY (id_mecanico, id_orden),

    CONSTRAINT fk_mo_mecanico
        FOREIGN KEY (id_mecanico) REFERENCES mecanico(id_mecanico)
        ON DELETE RESTRICT,

    CONSTRAINT fk_mo_orden
        FOREIGN KEY (id_orden) REFERENCES orden_servicio(id_orden)
        ON DELETE CASCADE
);

-- 10. Refacción
CREATE TABLE refaccion (
    id_refaccion      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    codigo_parte      VARCHAR(50)  NOT NULL,
    nombre_refaccion  VARCHAR(150) NOT NULL,
    precio_unitario_catalogo NUMERIC(12,2) NOT NULL CHECK
(precio_unitario_catalogo >= 0),
    existencias       INTEGER       NOT NULL DEFAULT 0 CHECK
(existencias >= 0),
    created_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),
    updated_at        TIMESTAMPTZ  NOT NULL DEFAULT NOW(),

```

```

        CONSTRAINT uq_refaccion_codigo UNIQUE (codigo_parte)
    );

CREATE INDEX idx_refaccion_codigo ON refaccion (codigo_parte);

-- 11. Detalle de orden (PK propia)
CREATE TABLE detalle_orden (
    id_detalle          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_orden            BIGINT          NOT NULL,
    id_refaccion        BIGINT          NOT NULL,
    cantidad            INTEGER         NOT NULL CHECK (cantidad > 0),
    precio_unitario_cobrado NUMERIC(12,2) NOT NULL CHECK
    (precio_unitario_cobrado >= 0),
    created_at          TIMESTAMPTZ     NOT NULL DEFAULT NOW(),

    CONSTRAINT fk_detalle_orden
        FOREIGN KEY (id_orden) REFERENCES orden_servicio(id_orden)
        ON DELETE CASCADE,

    CONSTRAINT fk_detalle_refaccion
        FOREIGN KEY (id_refaccion) REFERENCES refaccion(id_refaccion)
        ON DELETE RESTRICT
);

-- =====
-- Vista: costo total por orden (reemplaza el campo costo_total)
-- =====
CREATE VIEW v_costo_orden AS
SELECT
    id_orden,
    COALESCE(SUM(cantidad * precio_unitario_cobrado), 0) AS costo_total
FROM detalle_orden
GROUP BY id_orden;

-- =====
-- CONSULTAS DE EJEMPLO (INNER JOIN y LEFT JOIN)
-- =====

```